



Práctica 2. Herencia y Polimorfismo

Objetivos de aprendizaje:

1. Comprender y aplicar el concepto de herencia en Java	Entender qué es una jerarquía de clases y ser capaz de crear jerarquías que reutilizan y amplían el código mediante la herencia, asignando adecuadamente responsabilidades a cada clase.
2. Entender y utilizar el polimorfismo	Entender qué es el polimorfismo y demostrar su uso mediante métodos polimórficos y su redefinición en las clases derivadas, de manera que cada clase asuma su responsabilidad específica en el diseño.
3. Implementar métodos abstractos y clases abstractas	Comprender cómo se declaran e implementan métodos abstractos y el papel de las clases abstractas, responsabilidades comunes que deben concretarse en las subclases.
4. Reforzar las buenas prácticas de encapsulamiento y modularidad	Aplicar correctamente la ocultación de la información y la organización del código en archivos y paquetes según las convenciones de Java, favoreciendo un reparto claro de responsabilidades.
5. Convenciones de codificación en Java	Aplicar adecuadamente las normas de estilo y convenciones de codificación de Java (nombres de variables, formato, comentarios, indentación, etc.), mostrando código claro y legible.

Descripción

1. Primera Parte

En este ejercicio se va a implementar, a partir del código revisado de la práctica anterior, un juego basado en vidas en el que el usuario tenga que adivinar un número que conoce el programa. El código correspondiente a cada clase que se implemente deberá estar en ficheros separados con el mismo nombre que la clase.

1.1. Modificar la clase **JuegoConVidas**:

- Redefinir el atributo **record** para que se convierta en una variable no estática.
- Convertir el método **muestraVidasRestantes** en un método privado.
- Añadirle un método abstracto **juega** que tome como parámetro una cadena de caracteres (String) y que retorna un booleano.
- Ahora la clase **JuegoConVidas** pasa a ser una clase abstracta por lo que ya no se podrán crear instancias de ésta.

1.2. Crear la clase **JuegoAdivinaNumero**

- Deriva de la clase **JuegoConVidas** (Fig1).
- Tiene un constructor que toma dos parámetros de tipo entero. El primero es el número de vidas que, a su vez, se lo pasará al constructor de la superclase. El segundo parámetro representa el número a adivinar y está acotado entre 0 y 10.
- Implementa un método booleano denominado **validaNumero** que toma como parámetro un número entero. Este método retorna true si el número introducido es un número entre 0 y 10. En caso contrario, muestra por consola un mensaje indicando que el número está fuera de ese intervalo y retorna false.
- Implementa el método **juega**:
 - ✓ Convierte en un entero la cadena de caracteres (String) recibida como parámetro.
 - ✓ Valida si el número está comprendido entre 0 y 10. Para ello, usa la función **validaNumero** creada anteriormente. Si el número está fuera de rango, **juega** retorna true (y se sigue jugando). En caso de que se cumpla, se comprueba si el número recibido por parámetro coincide con el número a acertar:
 - Si son iguales: a) muestra por consola el mensaje: "Acertaste!!", b) actualiza la variable record y c) retorna false.
 - Si es diferente, comprueba si el número a acertar es mayor o menor que el que se pasó por parámetro mostrando por consola dicha información. A continuación, el método **juega** retorna el booleano obtenido de invocar el método **quitaVida** heredado.

1.3. Uso de la clase **MyInput**

Para la entrada de datos por teclado, se hará uso de la clase **MyInput**, que se puede descargar de la plataforma Moodle del curso. Esta clase deberá estar ubicada en otro paquete del mismo proyecto denominado "Entrada", que deberá ser importado por aquellas clases que lo utilicen.

1.4. Crear la clase principal

Contiene el método main donde:

1. Se crea una instancia de la nueva clase **JuegoAdivinaNumero**.
2. Se invoca el método **reiniciaPartida**.
3. Se define un bucle donde se solicita al usuario que introduzca un número entero de 0 a 10, y cuya salida viene gobernada por el valor de retorno del método **juega**.

2. Segunda Parte

Al código anterior se le añadirán dos juegos más, uno donde se adivinen números pares y otro de números impares.

2.1. Crear la clase **JuegoAdivinaNumeroPar**

- Deriva de la clase **JuegoAdivinaNumero** (Fig1).
- Redefine el método **validaNumero**. Este método vuelve a validar si el número

está entre 0 y 10 y a continuación comprueba si el número es par devolviendo true. Si el número es impar, muestra un mensaje de error por pantalla y devuelve false.

2.2. Crear la clase **JuegoAdivinaNumeroImpar**

- Deriva de la clase **JuegoAdivinaNumero** (Fig1).
- Redefine el método **validaNumero**. Este método vuelve a validar si el número está entre 0 y 10 y a continuación comprueba si el número es impar devolviendo true. Si el número es par, muestra un mensaje de error por pantalla y devuelve false.

2.3. Modificar la clase principal

- Crea un método estático denominado **jugar** que no retorna nada y recibe como parámetro una referencia de tipo **JuegoConVidas**. Dentro de este método se invoca el método **reiniciaPartida** a través de la referencia del tipo **JuegoConVidas** y a continuación, se define un bucle donde se solicita al usuario que introduzca un número entero de 0 a 10, y cuya salida viene gobernada por el valor de retorno del método **juega**.
- El método main crea una instancia de cada uno de los tres juegos creados:
 JuegoAdivinaNumero,
 JuegoAdivinaNumeroPar y
 JuegoAdivinaNumeroImpar.
 El número de vidas de cada juego será 5 y como número a adivinar un número cualquiera, otro par y otro impar respectivamente, todos comprendidos entre el 0 y el 10.
- Invocar el método **jugar** tres veces, pasándole como parámetro cada una de las instancias creadas.
- Ejecutar el programa y comprobar el funcionamiento de cada juego.



Fig. 1: Jerarquía de clases resultante

3. Evaluación y retroalimentación

3.1. Rúbrica de logro de resultados de aprendizaje

Criterio	Consigue el resultado	No consigue el resultado
1. Comprensión y aplicación de la herencia	Implementa correctamente jerarquías de clases, reutilizando y ampliando código mediante herencia; justifica el diseño y la asignación de responsabilidades con ejemplos del propio código.	No implementa jerarquías de clases o lo hace incorrectamente; no justifica ni comprende su utilidad.
2. Uso adecuado del polimorfismo	Utiliza métodos polimórficos y demuestra la redefinición en clases derivadas; explica con claridad cómo el polimorfismo simplifica o mejora el diseño y contribuye a una distribución coherente de responsabilidades.	No utiliza métodos polimórficos o no comprende la redefinición en clases derivadas; no sabe explicar su utilidad en el diseño.
3. Implementación de métodos y clases abstractas	Declara e implementa correctamente métodos abstractos y comprende el papel de las clases abstractas, justificando su necesidad en la solución propuesta y su función en la definición de responsabilidades comunes.	No implementa correctamente métodos/clases abstractas o no comprende su propósito en el diseño.
4. Encapsulamiento y modularidad	Aplica buenas prácticas de ocultación de información y organización modular del código (archivos, paquetes); justifica sus decisiones de diseño con ejemplos, mostrando un reparto claro de responsabilidades que favorezca la extensibilidad.	No aplica correctamente el encapsulamiento o la modularidad o no comprende su importancia.
5. Convenciones de codificación en Java	Aplica correctamente las normas de estilo Java y justifica sus decisiones mostrando comprensión de su importancia para la legibilidad y el mantenimiento del código.	No aplica correctamente las normas de estilo o no demuestra comprender su finalidad.

3.2. Preguntas de autoevaluación

Responde a las siguientes preguntas, justificando tus respuestas con fragmentos de código o ejemplos concretos desarrollados en la práctica:

1. Explica por qué se puede establecer una jerarquía de clases en esta práctica y cómo se aprovecha en este caso.
2. ¿Por qué se ha convertido JuegoConVidas en una clase abstracta? Explica la función y utilidad del método abstracto juega en la solución desarrollada.
3. ¿De qué manera la utilización del polimorfismo y la herencia en esta práctica ha permitido refactorizar o simplificar el código respecto a una solución sin

jerarquía de clases? Explica con ejemplos/contraejemplos concretos de tu código.

4. ¿En qué se diferencia una solución basada en Programación Orientada a Objetos (POO) de una que solo usa funciones o procedimientos?
5. ¿Por qué se dice que la POO permite crear programas más modulares y fáciles de mantener?
6. Pon un ejemplo de la práctica en el que la arquitectura de clases te haya ayudado a organizar mejor el código.
7. Explica cómo repartir las responsabilidades entre diferentes clases puede facilitar el mantenimiento o la ampliación del programa.

3.3. Obteniendo retroalimentación de la IAG

A continuación, deberás solicitar a la IAG con la que trabajes una primera evaluación (retroalimentación) sobre la corrección de tu código y las respuestas que has dado a las preguntas planteadas. Para ello elabora un prompt lo más detallado posible que permita a la IAG realizar una evaluación de la corrección tanto del código como de las respuestas proporcionadas. Es importante que en tu prompt incluyas:

- El enunciado de la práctica
- Los objetivos de aprendizaje
- La rúbrica de logro de resultados de aprendizaje.
- El código desarrollado
- Las preguntas de autoevaluación junto con tus respuestas

La valoración recibida **no debe ser una nota numérica**, sino un análisis en el que se indiquen:

- Qué aspectos están bien resueltos.
- Qué errores o carencias se detectan.
- Qué aspectos son mejorables.
- Si se han alcanzado los objetivos y resultados de aprendizaje previstos, tanto en el código como en las respuestas teóricas.

Es fundamental que tu prompt recoja con claridad todos los aspectos mencionados, de modo que la IAG oriente su análisis exactamente a lo que se espera en la evaluación. Si la respuesta de la IAG no es suficientemente clara, ajustada o útil, mejora tu prompt añadiendo explicaciones adicionales, ejemplos concretos o aclaraciones sobre lo que necesitas.

Recuerda: **la calidad de la retroalimentación recibida depende de la precisión y el contexto que incluyas en tu consulta**. Compara siempre la respuesta de la IAG con los materiales de clase u otras fuentes fiables, o consulta con el/la profesor/a en caso de dudas persistentes. Esto te ayudará a desarrollar un uso crítico y responsable de la IAG.

4. Informe que debe subirse al campus virtual de la asignatura

Una vez obtengas una respuesta satisfactoria, sube a la tarea correspondiente en Moodle un informe en PDF, **utilizando para ello la plantilla que está disponible en el campus virtual**. Este informe incluye:

- Nombre y apellidos
- La IAG utilizada, su versión (si procede) y la fecha.

- El desarrollo de la práctica: El código de la práctica y las respuestas a las preguntas que se formulan en esta práctica.
- Interacción con la IAG: Transcribe el diálogo íntegro y ordenado (tus prompts y las respuestas de la IAG).
- Una valoración personal sobre la utilidad y limitaciones de la retroalimentación recibida de la IAG, indicando qué mejoras podrás incorporar gracias a ella (si las hubiera) y reflexionando brevemente sobre su impacto en tu aprendizaje.
- La declaración de autoría y uso de IAG, conforme a lo indicado en el siguiente apartado.

Puedes aprovechar las observaciones de la IAG para mejorar el código o tus respuestas en la práctica siguiente.

Nota: Es obligatorio entregar el informe en PDF utilizando la plantilla que se facilita en el campus virtual de la asignatura y completar correctamente todos sus apartados; cualquier entrega que no se ajuste a esta plantilla —por ejemplo, subir solo el código o un PDF que no se corresponda con la plantilla— se considerará no válida y conllevará el suspenso de toda la práctica guiada (las cinco entregas).

5. Uso de la IAG, declaración de autoría y buenas prácticas

El uso de herramientas de IAG está permitido en esta práctica únicamente para obtener retroalimentación y sugerencias de mejora sobre el trabajo realizado (por ejemplo, evaluación de código, propuestas de corrección o ayuda en la redacción final). No se permite la elaboración inicial de código, respuestas o informes previos mediante IAG. Todo el desarrollo y resolución de la práctica debe ser realizado personalmente por el estudiante.

Para asegurar la transparencia, la responsabilidad y el uso adecuado de la IAG, la plantilla para la elaboración del informe incluye un apartado final de “Declaración de autoría y uso de IAG”. Mediante esta declaración, el estudiante manifiesta que el trabajo presentado ha sido realizado por él/ella, que asume la responsabilidad sobre su contenido y sobre las decisiones adoptadas durante su elaboración, y que la información proporcionada en el informe acerca del uso de herramientas de Inteligencia Artificial Generativa es completa y veraz.

La entrega del informe supone la aceptación expresa de esta declaración y la asunción de responsabilidad sobre el trabajo presentado.

Recuerda: El uso responsable de la IAG contribuye al aprendizaje autónomo y a la mejora continua, pero no puede sustituir al desarrollo personal de competencias técnicas y críticas. El falseamiento de la autoría será considerado una mala praxis académica.